2025 ICT 융합 프로젝트 공모전 결과보고서

1. 작품 제목

Monitor ARM

2. 작품 개요

각도 조절 가능한 모니터 로봇 팔은 사용자가 원하는 최적의 화면 위치를 자유롭게 설정할 수 있도록 하여 작업 효율성을 높이고, 장시간 작업 시 인체공학적인 자세를 유지할 수 있게 돕기 위해 제작되었습니다. 이를 위해 Mobile Aloha Viper X 로봇 팔 제어 시스템 구조를 기반으로, 사용자 맞춤형 조작이 가능한 시스템을 설계하여 정밀한 제어가 가능하도록 하였습니다.

3. 작품 설명

3-1-1 주요 동작 및 특징

이 모니터 암은 사용자가 직접 조이스틱 스위치를 이용하여 각 모터의 각도를 세밀하게 조정할 수 있는 기능을 제공합니다. 각 모터는 사용자의 입력에 따라 실시간으로 움직이며, 사용자는 모니터의 높이와 각도를 자유롭게 설정할 수 있습니다. 설정한 각도를 저장한 후, 코드상에 저장된 값을 통해 자동으로 각도를 조정할 수 있어 편리한 사용 경험을 제공합니다. 모터 제어는 라즈베리파이 통신을 통해 이루어지며, 조이스틱을 통해 각 모터에 전송되는 신호를 받아 모니터 암을 조정합니다. 이를 통해 사용자 맞춤형 작업 환경을 제공하고, 장시간 작업 시 편안한 자세를 유지할 수 있도록 돕습니다.

3-1-2 차별화된 기능

정밀한 모터 제어: 사용자는 SPI 통신을 통해 MCP 3008 아날로그-디지털 변환기를 이용하여 조이스틱의 입력 신호를 처리하고, 이를 기반으로 각 모터의 각도를 정밀하게 조정할 수 있습니다. 또한, 화면의 위치와 높이를 자유롭게 설정할 수 있어, 개인에게 최적화된 작업 환경을 구축할 수 있습니다. 이를 통해 사용자는 작업의 성격이나 신체 조건에 맞게 모니터를 최적의 위치로 배치할 수 있으며, 다양한 작업 스타일에 유연하게 대응할 수 있습니다. 이와 같은 세밀한 조정 기능은 사용자가 장시간 작업 시에도 편안한 자세를 유지할 수 있게 해 주며, 물리적인 피로를 줄여 작업 효율성을 극대화합니다. 또한, 각도 조정은 개인의 시선과 눈의 피로도를 최소화하는 방향으로 사용자가 설정할 수 있어, 장기적으로 건강한 작업 환경을 제공합니다.

각도 저장 및 자동 재설정: 조이스틱 버튼을 통해 사용자 맞춤형 제어 환경을 제공합니다. 저장된 방향 및 각도 값은 전원이 꺼졌다가 다시 켜져도 유지되므로, 모니터 암의 초기 상태를 재설정할 필요 없이 즉시 작업을 재개할 수 있습니다. 이 기능은 반복적인 작업 환경에서 동일한 설정값을 손쉽게 재사용할 수 있도록 하여 작업 효율성을 극대화하고 시간을 절약할 수 있습니다. 사용자가 원하는 설정을 직관적으로 저장하고 복원할 수 있는 이 기술은 사용자 친화적 인터페이스와 자동화된 작업 환경 구축에 유용합니다.

라즈베리파이 기반 다이나믹셀 제어: 라즈베리파이와 U2D2 를 결합하여 다이나믹셀 모터를 제어함으로써, 소형화된 하드웨어 환경에서도 고성능 모터 제어가 가능하도록 설계했습니다. U2D2 로 RS-485 네트워크를 활용하여 USB 연결을 통해 라즈베리파이와 다이나믹셀 간의 간단하고 효율적인 데이터 전송을 제공하며 각도 제어와 정밀한 움직임을 구현할 수 있고 이는 모니터 암의

복잡한 동작을 실현하는 데 최적화하였습니다.

3-1-3 사용자 편의성

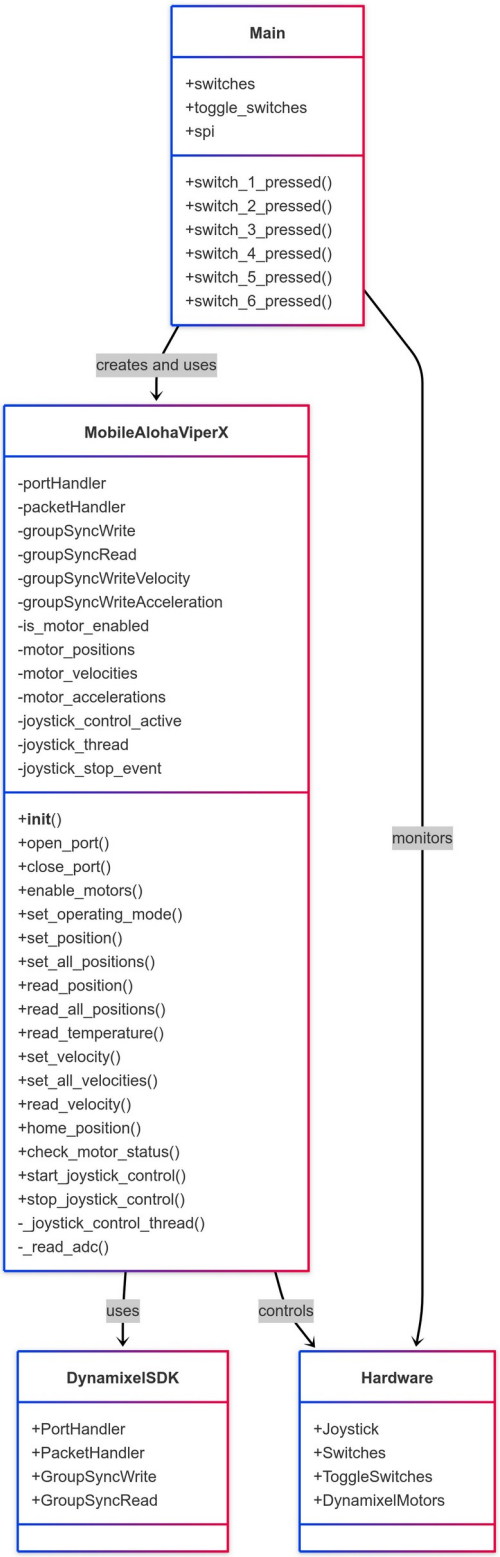
간편한 조작: 조이스틱 스위치를 이용한 직관적인 모터 제어는 사용자에게 빠르고 쉽게 모니터 각도를 조정할 수 있도록 돕습니다. 복잡한 설정 없이 손쉽게 원하는 위치로 모니터를 조정할 수 있습니다.

인체공학적 설계: 사용자가 모니터 각도를 자유롭게 조정함으로써 장시간 작업에도 편안한 자세를 유지할 수 있도록 돕습니다. 이를 통해 작업 중 발생할 수 있는 피로와 불편을 최소화합니다.

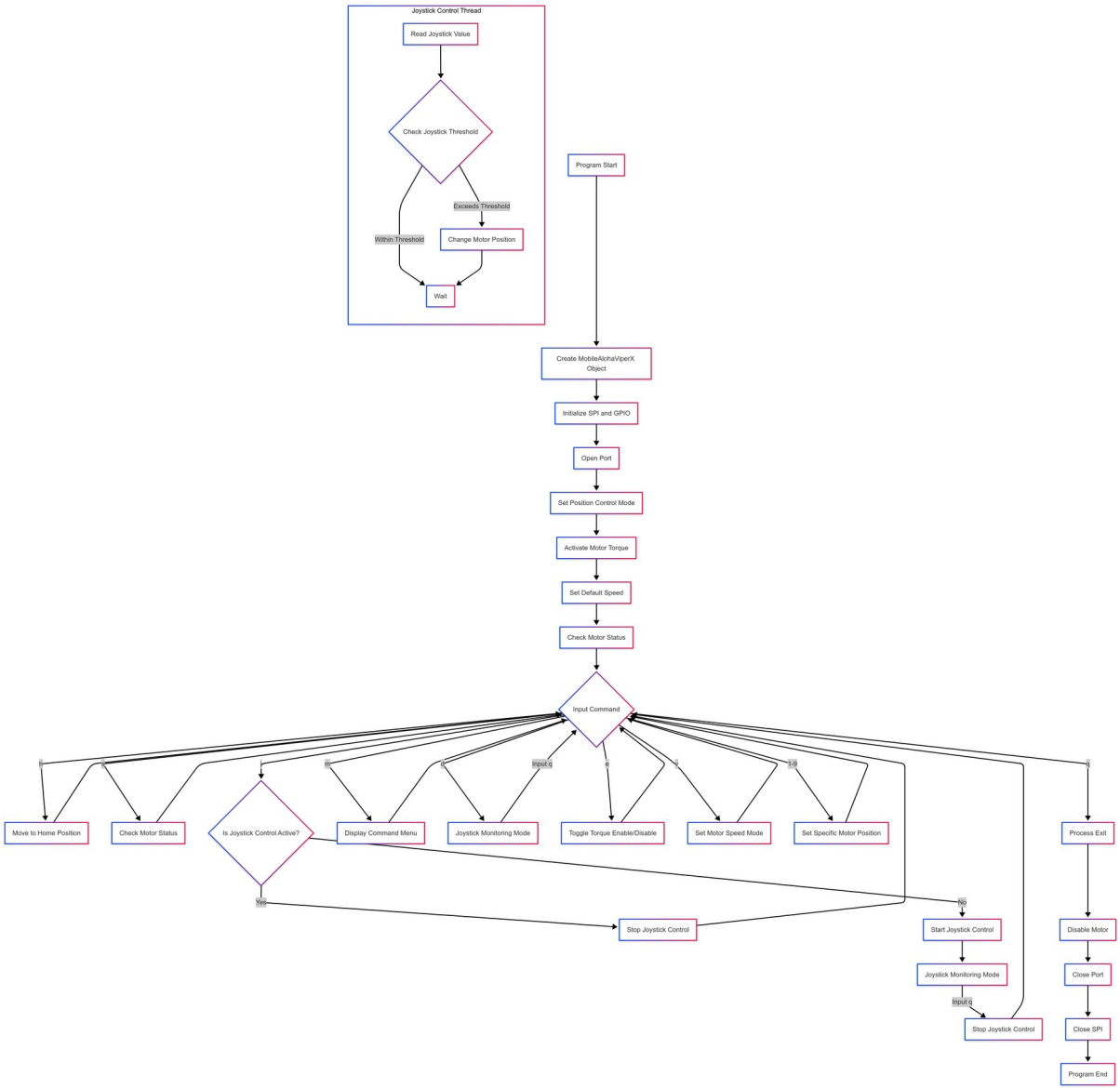
버튼 하나로 자동화: 버튼 하나로 저장된 각도를 불러오는 기능은은 사용자가 자주 사용하는 작업 환경을 쉽게 설정할 수 있도록 도와주어 사용성을 더욱 향상시킵니다.

3-2 전체 시스템 구성

3-2-1 제어 시스템 구성 요소



3-2-2 Monitor ARM 제어 흐름도



3-3 개발 환경

개발 언어: Python

개발 도구: Visual Studio Code

사용 시스템: Linux

4. 단계별 제작 과정

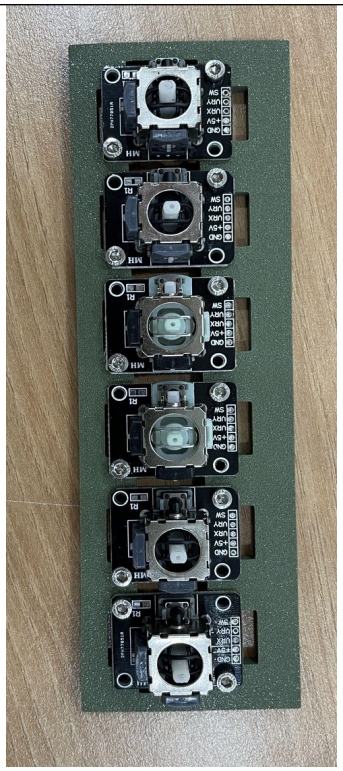
주제	추진날짜	추진 내용

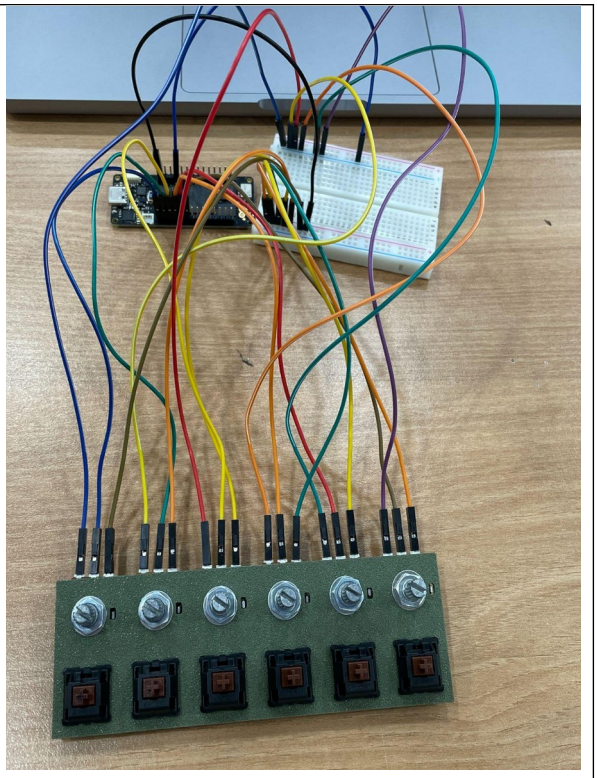
아이디어 구상	2/1 ~ 2/5	-프로젝트 목표 설정 및 기본 아이디어 도출 -제품의 주요 기능 정의 (모터 제어, 각도 저장 및 자동 제어 등) -사용자 요구 사항 분석 및 초기 설계 방향 설정
설계	2/6 ~ 2/12	-모니터 암의 구조 설계 및 모터 배치 계획 -라즈베리파이와 모터 연결 방식 및 제어 시스템 설계 -각도 조정 및 저장을 위한 소프트웨어 아키텍처 설계 -각 부품 및 재료 조사, 구매 계획 수립
구현	2/13 ~ 3/10	-하드웨어 조립: 모니터 암 구조물 제작 및 모터 설치 -라즈베리파이를 이용한 모터 제어 시스템 구축 -각도 저장 및 자동 제어 기능 개발 -사용자가 조작할 수 있는 인터페이스(조이스틱, 버튼) 개발 -초기 소프트웨어 구현 및 통합
테스트	3/11 ~ 3/20	-모니터 암의 각도 제어 기능 및 저장/불러오기 테스트 -하드웨어 및 소프트웨어의 안정성 확인 -사용자 경험 테스트 및 피드백 수집 -각도 조정 및 자동화 기능의 정확도 및 반응 속도 점검
최종 검토 및 보고서 작성	3/21 ~ 3/31	-모든 기능 점검 및 버그 수정 -최종 제품 테스트 및 품질 관리 -사용 설명서 작성 및 제품 패키징 준비 -프로젝트 보고서 작성 및 최종 검토

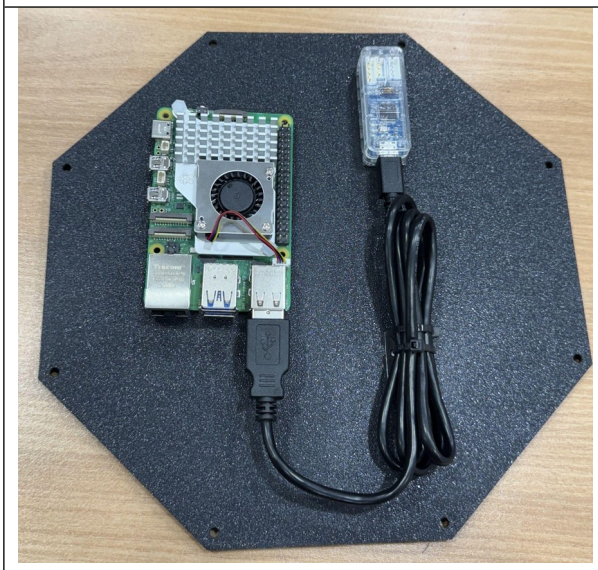
5. 사용한 제품 리스트

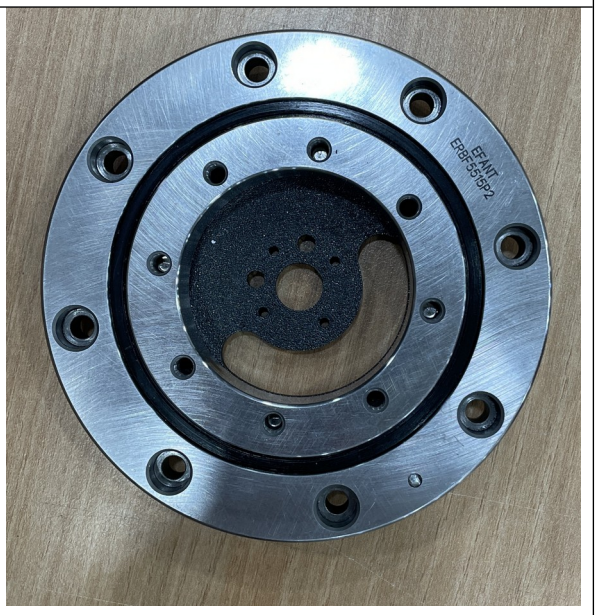
번호	상품코드	상품명(옵션명)	링크
1	12500010	2004 I2C 캐릭터 LCD (화이트/블루)	http://www.devicemart.co.kr/goods/view?no=12500010
2	1321195	테스트[CH254] 소켓 점퍼 케이블 40P (칼라) (M/F) 20cm	http://www.devicemart.co.kr/goods/view?no=1321195
3	15316634	라즈베리파이 5 호환 Standard PD45W 5V 5A 아답터 (C to C 케이블 포함)	http://www.devicemart.co.kr/goods/view?no=15316634
4	1322885	MCP3008-I/P	http://www.devicemart.co.kr/goods/view?no=1322885
5	30734	Toggle 2 단 3P MTS-102	http://www.devicemart.co.kr/goods/view?no=30734
6	15215449	라즈베리파이 5 (Raspberry Pi 5) 8GB + 가이드북	http://www.devicemart.co.kr/goods/view?no=15215449
7	1287087	아두이노 PS2 조이스틱 모듈 [SZH-EK056]	http://www.devicemart.co.kr/goods/view?no=1287087
8	11484	U2D2	http://www.devicemart.co.kr/goods/view?no=11484
9	13201647	다이나믹셀 액츄에이터 XH540-V270-R	https://www.devicemart.co.kr/goods/view?no=13201647
10	13201647	다이나믹셀 액츄에이터 XH430-V350-R	https://www.devicemart.co.kr/goods/view?no=13201634

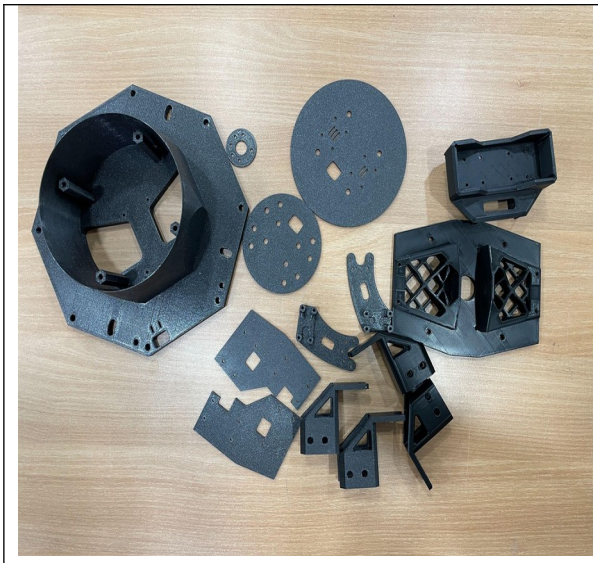
5. 결과물

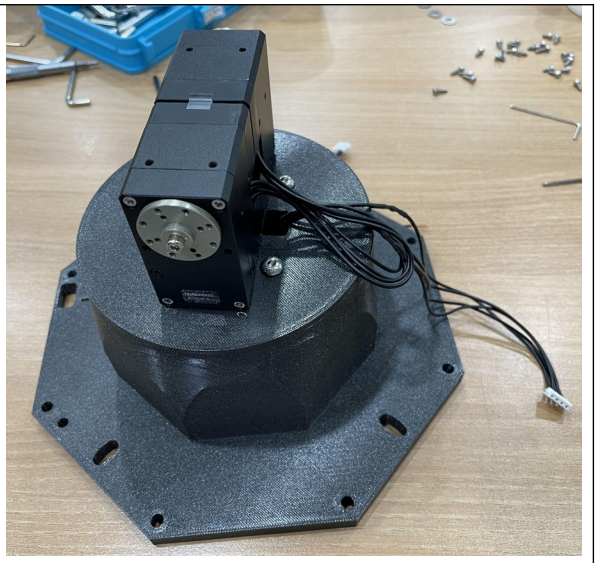







EPMT
ER8F5-1522

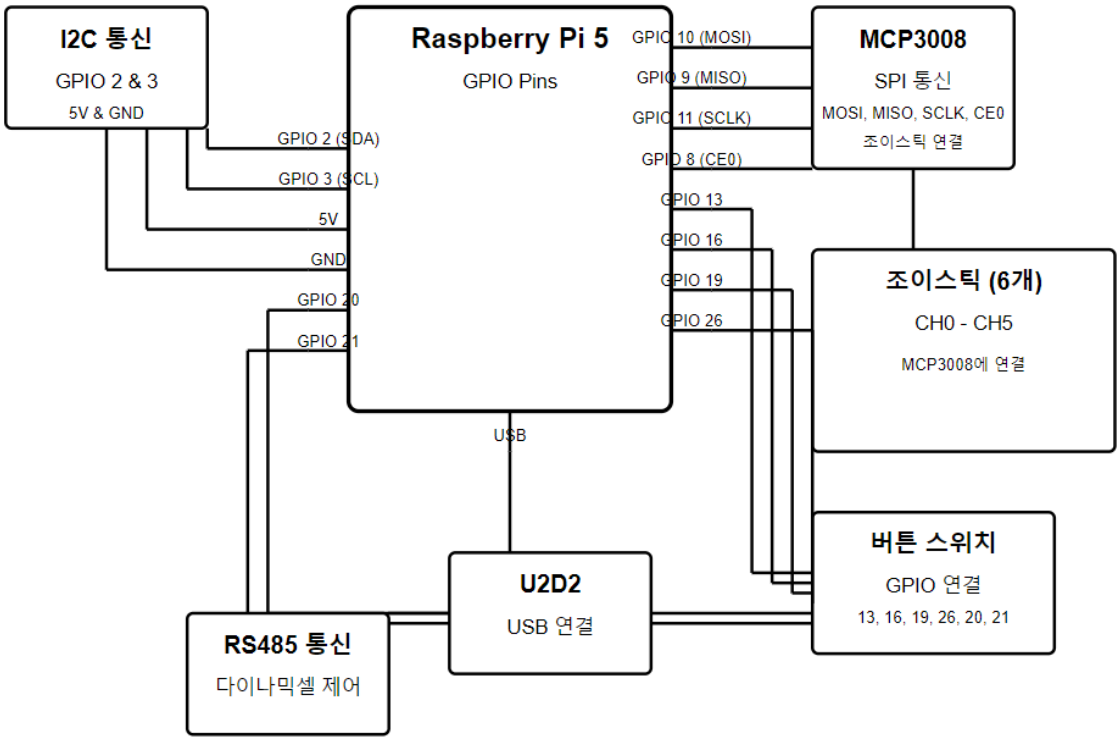




Edit
On
Preset
Off
M1
M2
M3
M4
M5
M6

6. 기타

6-1 회로도 (알고리즘)



6-2 소스코드

6-2 소스코드

Monitor ARM 로봇 팔 제어 시스템 상세 분석

1. 개요

Monitor ARM 은 다이나믹셀 모터를 활용한 로봇 팔 기반 모니터 암 제어 시스템입니다. 본 절에서는 Python 기반 제어 코드의 구조와 주요 기능을 정리합니다.

1.1 시스템 구성

Mobile Aloha Viper X 로봇 팔은 다음과 같은 하드웨어로 구성됩니다.

다이나믹셀 모터: ID 1-7 은 XH540-V270-R, ID 8-9 는 XH430-V350-R

조이스틱: MCP3008 ADC 를 통한 아날로그 입력 처리

버튼 및 스위치: GPIO 핀을 통한 디지털 입력 처리

통신 인터페이스: UART 기반 시리얼 통신을 통한 모터 제어

2. 다이나믹셀 모터 시스템

2.1 통신 프로토콜

다이나믹셀과의 통신은 ROBOTIS Dynamixel SDK 를 사용하며, 프로토콜 2.0 과 57600 bps 를 기준으로 설정됩니다.

```
PROTOCOL_VERSION = 2.0
BAUDRATE = 57600
DEVICENAME = '/dev/ttyUSB0'
```

디바이스 경로는 Linux 기준 '/dev/ttyUSB0'입니다.

2.2 제어 테이블 주소

모터 제어와 상태 조회를 위해 다음과 같은 제어 테이블 주소를 사용합니다.

```
ADDR_OPERATING_MODE = 11
ADDR_TORQUE_ENABLE = 64
ADDR_LED_RED = 65
ADDR_GOAL_POSITION = 116
ADDR_GOAL_VELOCITY = 104
ADDR_PRESENT_POSITION = 132
ADDR_PRESENT_VELOCITY = 128
ADDR_PRESENT_CURRENT = 126
ADDR_PRESENT_TEMPERATURE = 146
ADDR_PROFILE_VELOCITY = 112
ADDR_PROFILE_ACCELERATION = 108
```

3. 시스템 아키텍처

3.1 클래스 구조

전체 제어 시스템은 MobileAlohaViperX 클래스로 구성되어 있습니다.

주요 기능은 포트 및 패킷 핸들러 초기화, 모터 토크 제어, 위치/속도 설정, 상태 모니터링, 조이스틱 입력 처리입니다.

3.2 초기 설정 예시

```
def __init__(self):
    self.portHandler = PortHandler(DEVICENAME)
    self.packetHandler = PacketHandler(PROTOCOL_VERSION)
    self.groupSyncWrite = GroupSyncWrite(
        self.portHandler, self.packetHandler, ADDR_GOAL_POSITION, 4
    )
    self.groupSyncRead = GroupSyncRead(
        self.portHandler, self.packetHandler, ADDR_PRESENT_POSITION, 4
    )
    self.is_motor_enabled = False
    self.motor_positions = {}
```

4. 모터 제어 및 동기화

4.1 모터 ID 및 동기화 설정

```
DXL_ID_LIST = list(range(1, 10))
XH540_ID_LIST = list(range(1, 8))      # IDs 1-7
XH430_ID_LIST = list(range(8, 10))     # IDs 8-9

SYNC_PAIRS = {
    2: 3,    # 2번 모터(마스터)와 3번 모터(슬레이브)
    4: 5,    # 4번 모터(마스터)와 5번 모터(슬레이브)
}
MASTER_MOTORS = [2, 4]
SLAVE_MOTORS = [3, 5]
```

4.2 영점 및 제한 범위

```
DEFAULT_POSITION = 2048
ZERO_POSITIONS = {
    2: 1950,
    4: 2060,
    6: 2429,
    7: 2304,
    8: 1536,
}

MOTOR_LIMITS = {
    2: {'min': 1950, 'max': 4095},
    4: {'min': 1987, 'max': 4095},
    7: {'min': 0, 'max': 2674},
}
```

4.3 위치 제어

```
def set_position(self, motor_id, position):
    if motor_id in MOTOR_LIMITS:
        limits = MOTOR_LIMITS[motor_id]
        if position < limits['min']:
            position = limits['min']
        elif 'max' in limits and position > limits['max']:
            position = limits['max']

    positions_to_set = {motor_id: position}
    if motor_id in MASTER_MOTORS and motor_id in SYNC_PAIRS:
        slave_id = SYNC_PAIRS[motor_id]
        positions_to_set[slave_id] = position

    # 동기화된 모터에 목표 위치 전송
```

5. 조이스틱 제어 시스템

5.1 조이스틱 설정

```
JOYSTICK_THRESHOLD_LOW = 10
JOYSTICK_THRESHOLD_HIGH = 1000
JOYSTICK_MIDDLE = 512
JOYSTICK_STEP = 20

JOYSTICK_MOTOR_MAP = {
    'X축1': 1,
    'Y축1': 2,
    'Y축2': 4,
    'X축2': 6,
    'Y축3': 7,
    'X축3': 8,
}
```

5.2 ADC 인터페이스

```
def _read_adc(self, spi_device, channel):
    """MCP3008에서 채널 데이터를 읽는 함수"""
    if channel > 7 or channel < 0:
        return -1
    adc = spi_device.xfer2([1, (8 + channel) << 4, 0])
    value = ((adc[1] & 3) << 8) + adc[2]
    return value
```

5.3 조이스틱 제어 쓰레드

```
def start_joystick_control(self, spi_device):
    """조이스틱을 이용한 모터 제어 시작"""
    self.joystick_thread = threading.Thread(
        target=self._joystick_control_thread,
```

```
        args=(spi_device,),
        daemon=True,
    )
    self.joystick_thread.start()
```

6. 사용자 인터페이스 및 제어

6.1 키보드 입력 처리

```
print("\n====== 명령어 메뉴 ======")
print('h: 홈 위치(영점)로 이동')
print('s: 모터 상태 확인')
print('j: 조이스틱 제어 모드 시작/중지')
print('m: 수동 모터 제어 모드')
print('e: 토크 활성화/비활성화 전환')
print('q: 종료')

if os.name == 'nt':
    import msvcrt
    def getch():
        return msvcrt.getch().decode()
else:
    import sys, tty, termios
    fd = sys.stdin.fileno()
    old_settings = termios.tcgetattr(fd)
    def getch():
        try:
            tty.setraw(sys.stdin.fileno())
            ch = sys.stdin.read(1)
        finally:
            termios.tcsetattr(fd, termios.TCSADRAIN, old_settings)
        return ch
```

6.2 물리적 버튼 및 스위치

```
switch_pins = [13, 19, 26, 16, 20, 21]
toggle_switch_pins = [23, 24]
switches = {pin: Button(pin, pull_up=True) for pin in switch_pins}
```

7. 안전 기능 및 오류 처리

7.1 범위 제한 및 콜백

```
switch_1_pin = switch_pins[0]
def switch_1_pressed():
    print('스위치 1번이 눌렸습니다. 영점 위치로 이동합니다.')
    robot.home_position()
```

7.2 온도 모니터링

```
if temp > 70:
    print(f"⚠️ 경고: 모터 {motor_id} 과열 감지!")
```

7.3 예외 처리

```
try:
    if not robot.open_port():
        print('포트 열기 실패. 프로그램을 종료합니다.')
        exit(1)
except KeyboardInterrupt:
    print('\n프로그램이 사용자에 의해 중단되었습니다.')
except Exception as e:
    print(f"\n오류 발생: {e}")
finally:
    # 정리 작업
    pass
```

8. 시스템 작동 모드 및 상태

8.1 제어 모드

```
POSITION_CONTROL_MODE = 3
VELOCITY_CONTROL_MODE = 1
CURRENT_CONTROL_MODE = 0
PWM_CONTROL_MODE = 16
```

8.2 상태 모니터링

```
def check_motor_status(self):
    print("\n====== 모터 상태 확인 ======")
    all_ok = True
    for motor_id in DXL_ID_LIST:
        position = self.read_position(motor_id)
        temp = self.read_temperature(motor_id)
        velocity = self.read_velocity(motor_id)
        # 상태 표시 로직
```

9. 메인 루프 및 사용자 상호작용

9.1 메인 루프

```
while True:
    if monitoring_mode:
        # 조이스틱 값 모니터링 모드
        pass
    else:
        print("\n명령어를 입력하세요 (메뉴 보기: m): ", end="")
        key = getch()
        # 키 입력 처리
```

9.2 디버깅 및 모니터링

```
clear_screen()
print("======== 조이스틱 입력 모니터링 ========")
print("(q 키를 누르면 메뉴로 돌아갑니다)")
print("\n======== 아날로그 입력 ========")
for channel_info in analog_channels:
    value = robot._read_adc(spi, channel_info["channel"])
    motor_id = channel_info["motor_id"]
    print(f"{channel_info['name']} (모터 ID {motor_id}): {value}")
```

10. 결론 및 추가 개선 가능성

10.1 요약

본 시스템은 9 개의 다이나믹셀 모터 제어, 모터 간 동기화, 조이스틱 입력 처리, 물리적 버튼 및 스위치 제어, 온도 및 위치 기반 안전 기능을 제공합니다.

또한 객체 지향 구조로 설계되어 유지보수와 기능 확장이 용이합니다.

10.2 개선 가능성

보다 고급 모션 계획 알고리즘과 역운동학(inverse kinematics) 적용

사용자 정의 작업 시퀀스 저장 및 재생 기능 추가

LED, 음향, 햅틱 등 다양한 피드백 장치 연동

웹 또는 모바일 기반 원격 제어 인터페이스 구축

토크 기반 충돌 감지와 자동 복구 루틴 고도화